

ORGANIZAÇÃO DE COMPUTADORES I

Trabalho Prático 2

Daniel Augusto Lopes - 2022036160
Gustavo Dias Apolinario - 2022035911
Lorenzo Ventura Vagliano - 2022035938

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte - MG - Brasil

1 Introdução

O RISC-V é uma arquitetura de conjunto de instruções (ISA) de código aberto originalmente desenvolvido na Universidade da Califórnia, em Berkeley. A tarefa principal proposta neste trabalho prático foi incluir novas operações e módulos numa implementação do RISC-V cinco estágios feita na linguagem de descrição de hardware Verilog, rodando na plataforma Google Colab. Neste trabalho, as instruções por nós implementadas foram as operações de multiplicação (mul), divisão (div), “e” lógico bit a bit com um valor imediato (andi) e branch on equal (beq).

2 Desenvolvimento

2.1 Assembly

Na seção Assembly foram incluídas as codificações referentes a cada uma das novas operações implementadas. Se tratando das instruções mul e div, lhes definimos como operações do tipo R no instruction_format. Estipulamos um valor de opcode, funct3 e funct7 para ambas. As instruções beq e andi já contavam com esses valores pré-estabelecidos, e não foram alterados. Na tabela a seguir, estão registrados os respectivos valores para cada uma das operações.

Instrução	Tipo	Opcode	Funct3	Funct7
mul	R	0110011	001	0100011
div	R	0110011	010	0100011
andi	I	0010011	111	-
beq	B	1100011	000	-

2.2 Control Unit

A unidade de controle serve para decodificar a instrução recuperada, determinando qual operação deve ser realizada e quais operandos estão envolvidos. Dentro do case que lia qual o código da operação já estavam incluídos os opcode's para instruções do tipo R e tipo I, respectivamente com as tags add e addi. Assim, incluímos um novo case para detectar operações de tipo B, permitindo o reconhecimento da instrução beq, reproduzido a seguir:

```
7'b1100011: begin // beq == 99
    aluop <= 2'b1;
    ImmGen <= {{19{inst[31]}},inst[31],inst[7],inst[30:25],inst[11:8],1'b0};
    regwrite <= 1'b0;
    branch_eq <= 1'b1;
```

No caso onde o aluop seja 1100011, a unidade de controle reconhecerá a operação do tipo B, assinalará os valores do aluop e branch_eq como 1, e do regwrite como 0, já que não há escrita nos registradores. Além disso, fará o cálculo do valor do imediato utilizado no deslocamento.

2.3 ALU Control and ALU

Nesta seção do programa, foram feitas algumas alterações. Elas podem ser divididas em três partes principais: case(ctl) , case(funcnt [3:0]), case(funcnt [2:0]).

Na primeira parte “case(ctl)” houve a necessidade de se acrescentar as operações de multiplicação e divisão. Essas operações foram atribuídas aos valores 4'd10 e 4'd11 respectivamente. Esse processo foi preciso, pois a ALU precisa fazer essas operações a fim de fornecer o resultado correto nas instruções “mul” e “div”.

Na segunda parte, a fim de completar a ideia da primeira parte, foi preciso fazer uma conexão entre a instrução e a operação realizada. Sabendo que o funcnt [3:0] seria utilizado para as instruções “mul” e “div”, pois elas são do tipo-R, foram criados dois novos cases, um para cada instrução. Esses valores são 4'd9 e 4'd10, esse valor é reconhecido e computado da seguinte forma: o segundo bit mais significativo do funcnt_7 + funcnt_3. Isso quer dizer que se concatena o segundo bit mais significativo do funcnt_7 com todo o funcnt_3 da instrução, essa concatenação irá gerar um binário, sendo esse binário em decimal igual a 9 e 10 para as instruções “mul” e “div”, respectivamente. Por fim, quando esses cases 4'd9 e 4'd10 são encontrados, ou seja, verdadeiros, o _funcnt recebe 4'd10 e 4'd11 para terminar a conexão com a ALU.

Na terceira parte, case(funcnt[2:0]) foi realizada uma mudança focando no “andi”. Essa operação por ser uma instrução do tipo-I, não é processada da mesma forma que “mul” e “div”, pois elas não possuem um funcnt_7. Dessa forma, o case(funcnt[2:0]) é analisado

exclusivamente pelo valor do `funct_3`. No caso o “andi” possui o valor de 7 como seu `funct_3`, portanto foi acrescentado uma conexão entre o case 3’d7 com o `_functi` 4’d0, uma vez que esse valor de 4’d0 na ALU realiza a operação de “and desejada. Nessa perspectiva, a conexão entre a instrução “andi” e a ALU foi completada.

2.4 Compilando e Executando

Nesta seção, observou-se que caso o número de ciclos fosse muito baixo, como o de apenas 20 previamente definido, algumas operações enfrentariam problemas para serem concluídas. Assim, modificou-se o valor do clock para 60, o que sanou tais problemas.

3 Testes

Os testes aqui descritos foram todos executados na plataforma do Google Colab, podendo ser reproduzidos se necessário. É importante salientar que, no caso específico da instrução `beq`, deparamo-nos com problemas decorrentes da conversão errada das instruções em assembly para hexadecimal, feita pelo conversor do Colab. Fazendo a conversão das mesmas instruções para hexadecimal através do [venus simulator](#), os testes com o branch on equal funcionaram corretamente. Assim sendo, acredita-se que trata-se de um erro da implementação do compilador, e não da instrução em si.

3.1 mul

Instruções:

```
nop  
addi x1, x0, 6  
addi x2, x0, 2  
mul x3, x1, x2  
end: nop
```

Saída:

```
x0 = 0  
x1 = 6  
x2 = 2  
x3 = c (12 em hexadecimal)
```

Neste teste são adicionados os valores imediatos 6 e 2 aos registradores `x1` e `x2`, respectivamente. Então é realizada a multiplicação entre os valores nesses registradores, com o resultado sendo gravado em `x3`. A saída mostra que os valores foram escritos corretamente, e que obteve-se o resultado esperado da operação (12).

3.2 div

Instruções:

```
nop  
addi x1, x0, 6  
addi x2, x0, 2  
div x3, x1, x2  
end: nop
```

Saída:

```
x0 = 0  
x1 = 6  
x2 = 2  
x3 = 3
```

Neste teste são adicionados os valores imediatos 6 e 2 aos registradores x1 e x2, respectivamente. Dessa vez, é realizada a divisão entre os valores nesses registradores, com o resultado sendo novamente gravado em x3. A saída mostra que os valores foram escritos corretamente, e que obteve-se o resultado esperado da operação (3).

3.3 andi

Instruções:

```
nop  
addi x1, x0, 1  
addi x2, x0, 2  
andi x3, x1, 2  
andi x4, x2, 3  
end: nop
```

Saída:

```
x0 = 0  
x1 = 1  
x2 = 2  
x3 = 0  
x4 = 2
```

No teste do and lógico com imediato, são adicionados os valores imediatos 1 e 2 aos registradores x1 e x2, respectivamente. A seguir são feitas duas operações andi. Primeiro, é feito o “e” lógico bit a bit entre 01 - armazenado em x1 - e o valor imediato 2 (10), com o resultado sendo armazenado em x3. Essa operação entre 01 e 10 resultará em 00. Além disso, é feito o andi entre x2 e o valor imediato 3 (11), armazenando o resultado em x4. A operação entre 10 e 11 resultará em 10 (2 em binário). Avaliando a saída, com 0 em x3 e 2 em x4, nota-se que o “e” lógico está corretamente implementado.

3.4 beq

Instruções:

```
nop  
addi x1, x0, 4  
addi x2, x0, 4  
beq x2, x1, end  
sub x2, x2, x1  
end: addi x3, x0, 7
```

Saída:

```
x0 = 0  
x1 = 4  
x2 = 4  
x3 = 7
```

Neste caso de teste, é adicionado o valor imediato 4 à ambos os registradores x1 e x2, para em seguida realizar a instrução branch on equal entre estes dois registradores. Naturalmente, a igualdade será verdadeira e a execução irá saltar para o label end, que determina a operação de adição de 0 com o valor imediato 7, com a resposta escrita em x3. Conforme observa-se na saída, tal operação foi realizada com sucesso. Aqui é importante notar que a operação de subtração entre x1 e x2, com resposta gravada em x2, foi ignorada devido ao salto. Caso contrário, teríamos 0 em x2 na saída. Com isso, verifica-se que a verificação de igualdade e o salto estão funcionando conforme o esperado.

4 Conclusão

Ao fim do trabalho, os objetivos inicialmente estabelecidos foram alcançados. As quatro instruções requisitadas - mul, div, andi e beq - foram implementadas e testadas quanto ao seu funcionamento correto. Para fazê-lo, tivemos de familiarizar-nos com a linguagem de descrição de hardware Verilog para destrinchar o funcionamento do código base, que simula o processador RISC-V. Além das seções que não demandaram alterações, foi necessário analisar a fundo e entender o funcionamento em conjunto da unidade de controle e da ALU,

bem como as declarações e codificações em assembly, além das especificidades próprias de cada tipo de instrução, que valem-se de operandos distintos.

Para as operações aritméticas de multiplicação e divisão, estipulamos valores de opcode, funct3 e funct7, já que estes não vinham previamente definidos. Para a instrução de branch on equal, tivemos de definir um novo caso na unidade de controle referente às operações de tipo B. No ALU Control, tivemos de definir as novas operações aritméticas implementadas, bem como incluir novos cases que possibilitassem a designação de qual a operação a ser realizada a partir dos bits da instrução.

Durante a implementação deparamo-nos frequentemente com alguns erros de lógica e bugs, além do já anteriormente mencionado problema com o compilador para hexadecimal no caso da instrução beq, que tiveram de ser estudados e depurados.